

LINUX INTERGALÁCTICO: BASE DA
JORNADA JEDI

RUMO AO SERVIÇO WEB COM LAMP!



8

LISTA DE FIGURAS

Figura 1 – Building blocks para servidores LAMP	9
Figura 2 – Comparativo entre servidores	10
Figura 3 – Status do Apache	12
Figura 4 – Status do firewall local.....	13
Figura 5 – Página default do Apache	14
Figura 6 – Habilitação do virtual host cyberhints.com	18
Figura 7 – Liberação de portas adicionais.....	19
Figura 8 – Verificação dos estados das portas.....	20
Figura 9 – Página para testes - cyberhints.com	21
Figura 10 – Senha atual do root.....	22
Figura 11 – Troca da senha do root	23
Figura 12 – Remoção de usuários anônimos	23
Figura 13 – Desabilitando o login remoto	23
Figura 14 – Remoção do database test	24
Figura 15 – Verificando o status do serviço MariaDB.....	24
Figura 16 – Configuração do <i>MariaDB</i>	25
Figura 17 – Instalação do PHP.....	27
Figura 18 – Configurar banco de dados.....	30
Figura 19 – Senha para o phpMyAdmin.....	30
Figura 20 – Boas-Vindas PHPMyAdmin.....	31
Figura 21 – Painel principal do PHPMyAdmin.....	31

LISTA DE QUADROS

Quadro 1 – Especificações da VM do servidor LAMP	11
--	----

LAMP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Ajustes nas portas do Apache	20
Código-fonte 2 – Página para testes - cyberhints.com	21
Código-fonte 3 – Exemplo de código PHP	26
Código-fonte 4 – Arquivo index.php acessando o MariaDB	28

EXEMPLO

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 – Instalação do servidor APACHE2	12
Comando de prompt 2 – Verificação do status do serviço APACHE2.....	12
Comando de prompt 3 – Configuração do firewall local	13
Comando de prompt 4 – Habilitando e desabilitando módulos MPM	16
Comando de prompt 5 – Criação dos subdiretórios do virtual host cyberhints.com	17
Comando de prompt 6 – Arquivo de configuração do virtual host cyberhints.com ..	18
Comando de prompt 7 – Instalação do <i>MariaDB</i>	22
Comando de prompt 8 – Configuração do <i>MariaDB</i>	25
Comando de prompt 9 – Configuração básica do PHP	27
Comando de prompt 10 – Acesso ao index.php.....	28
Comando de prompt 11 – Inserção de repositórios.....	29

SUMÁRIO

RUMO AO SERVIÇO WEB COM LAMP!	7
1 <i>FREE AND OPEN SOURCE SOFTWARE</i>	7
1.1 <i>Software</i> livre	7
2 BLOCOS DE CONSTRUÇÃO DO LAMP	8
3 QUE COMECEM OS JOGOS!	10
4 CONFIGURAÇÕES DA MÁQUINA VIRTUAL (VM)	11
5 INSTALAÇÃO SERVIDOR APACHE	12
6 CRIANDO VIRTUAL HOSTS	14
6.1 Ajustando os módulos do APACHE	15
6.2 Criando os subdiretórios para os <i>virtual hosts</i>	17
6.3 Configurando e ativando os <i>virtual hosts</i>	17
6.4 Ajustando as configurações do <i>firewall</i>	19
6.5 Ajustando as configurações de portas do APACHE	19
7 INSTALAÇÃO DO MARIADB	21
8 INSTALAÇÃO DO PHP	25
REFERÊNCIAS	33

RUMO AO SERVIÇO WEB COM LAMP!

1 FREE AND OPEN SOURCE SOFTWARE

LAMP é um acrônimo para *Linux* – *APACHE* – *MySQL* - *PHP*, ou seja, uma plataforma completa para desenvolvimento e, dependendo do contexto, publicação de aplicações *web*. Uma das principais características dos servidores LAMP decorre do fato de serem totalmente compostos por FOSS: *free and open source software* (*software* livre e de código aberto).

1.1 Software livre

ATENÇÃO: **nunca** entenda o termo “*free*” da expressão ‘*free software*’ como grátis ou gratuito! Este “*free*” significa LIVRE!

Embora isso possa não parecer tão relevante em nosso contexto, pelo contrário, é fundamental. Com base na definição da *Free Software Foundation* (<https://www.fsf.org/>), tem-se:

Por ‘software livre’ devemos entender aquele software que respeita a liberdade e senso de comunidade dos usuários. Grosso modo, isso significa que os usuários possuem a liberdade de executar, copiar, distribuir, estudar, mudar e melhorar o software. Assim sendo, ‘software livre’ é uma questão de liberdade, não de preço. Para entender o conceito, pense em ‘liberdade de expressão’, não em ‘cerveja grátis’. Por vezes chamamos de ‘libre software’ para mostrar que livre não significa grátis, pegando emprestado a palavra em francês ou espanhol para “livre”, para reforçar o entendimento de que não nos referimos a software como grátis. (O QUE É, s.d., n. p.).

E como isto se aplica ao nosso contexto? Bem, considere que embora a estrutura original de um servidor LAMP recaia sobre a descrição dada no logo início deste capítulo, pode haver variações, como: o *MySQL* ser substituído pelo *MariaDB* e o *PHP* ser substituído pelo *Perl* ou mesmo pelo *Python*. Consideremos especificamente o caso do *MySQL* e *MariaDB* sob a ótica do licenciamento. O *MariaDB* é um *fork* (de foma simples, uma derivação) do *MySQL*. Porém, há diferenças significativas entre ambos acerca das quais nos limitaremos aqui ao fato de ser *MariaDB* licenciado sob os termos da GPLv2, enquanto o *MySQL* possui dois tipos de licenciamento: (I) GPLv2 – aplicada ao *MySQL Community Edition*, e (II) *Enterprise* – outro tipo de licenciamento orientado ao uso comercial do produto.

Resumidamente, a principal diferença entre os dois tipos de licenciamento para o *MySQL* recai sobre as funcionalidades e o suporte disponibilizados. Enquanto o *MariaDB* disponibiliza a seus usuários os pacotes de *software* completos, o *MySQL Community Edition* não contempla funcionalidades importantes como *Threadpool*. E o que significa? O *MySQL* cria uma *thread* para cada conexão ou instrução que executa. Portanto, ao exceder um certo número de *threads*, a performance começará a degradar. *Threadpool* é um recurso que permite ao servidor reduzir a quantidade de *threads*, e com isso diminui também a troca de contextos (dentre outras coisas), o que contribui para que o servidor mantenha seu melhor desempenho. Dessa forma, a falta deste recurso pode impactar negativamente a *performance* do banco de dados e de consultas (*queries*).

Ficou claro por que o tipo de licenciamento, por que o fato de ser *software* livre, pode ser tão relevante? Este aspecto pode refletir diretamente sobre a qualidade e viabilidade do projeto.

2 BLOCOS DE CONSTRUÇÃO DO LAMP

Servidores LAMP são sistemas que podem ser implementados por diferentes blocos de construção (*building blocks*), conforme ilustrado pela Figura “*Building blocks para servidores LAMP*”.

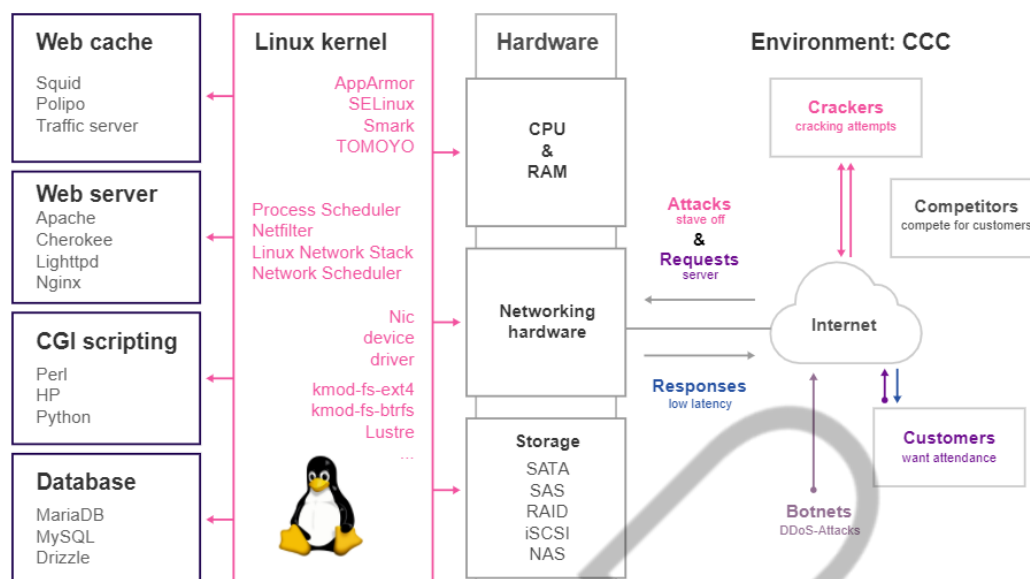


Figura 1 – Building blocks para servidores LAMP
Fonte: TRAIAN (2013)

Ela ilustra como a construção de um servidor LAMP pode ser feita de forma bastante alinhada às expectativas e necessidades do interessado. Iniciando-se pelo sistema operacional Linux, o qual pode ser encontrado sob forma de diferentes distribuições a exemplo do Debian, Ubuntu e Fedora dentre muitas outras.

Por diversas razões, essas distribuições são bastante recomendáveis à implementação de servidores LAMP, entretanto, algumas das mais relevantes – especialmente no contexto aqui considerado, recaem sobre: (I) facilidade na instalação e gerenciamento dos pacotes a serem utilizados e (II) variedade e qualidade da documentação disponível.

O servidor web mais utilizado em *LAMP servers* – e em função disto tem seu nome no acrônimo – é o APACHE, embora excelentes alternativas como o Cherokee e NGINX (lê-se “éngini ex”) estejam disponíveis. Em caso de implementação para produção ou em projetos com necessidades mais específicas, vale a pena acessar os projetos de cada um destes e se inteirar acerca de suas características e particularidades.

A Figura “Comparativo entre servidores” ilustra uma pesquisa feita pelo web site STACKSHARE acerca da preferência de diversos tipos de usuários em relação a estes três servidores.

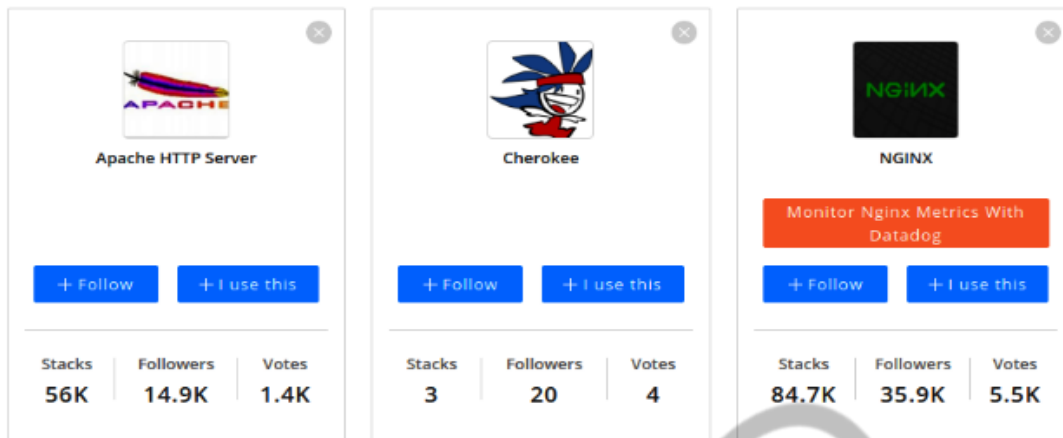


Figura 2 – Comparativo entre servidores
Fonte: STACKSHARE (2021)

Analogamente aos servidores *web*, os gerenciadores de banco de dados (*database management systems*) também oferecem excelentes alternativas para implementações LAMP.

Naturalmente, dentre estas, as mais difundidas são o *MySQL* e *MariaDB*. Embora a Figura “*Building blocks para servidores LAMP*” ainda referencie o *Drizzle*, o projeto deste gerenciador de banco de dados, também *fork* do *MySQL*, foi abandonado.

3 QUE COMECEM OS JOGOS!

Antes de iniciar a construção de nosso servidor LAMP, vale reforçar que este será implementado de forma básica, com objetivo de apresentar a plataforma e dar ideia de seu potencial. A discussão aprofundada sobre a configuração de cada serviço envolvido extrapola o escopo do nosso estudo neste momento, portanto, recomenda-se enfaticamente que, para fins de produção, cada um destes seja adequadamente estudado a fim de garantir segurança mínima à plataforma como um todo.

Em suma, podemos dizer que LAMP são servidores que permitem desenvolver e disponibilizar conteúdo *web* (tradicionalmente) via servidor *APACHE*, armazenar dados (tradicionalmente) via *MySQL* e implementar *server-side scripts* (tradicionalmente) por meio do PHP: linguagem para *script* de alto nível. O PHP é uma ferramenta amplamente utilizada para desenvolvimento de páginas *web*, geralmente, embutido em blocos de código HTML.

4 CONFIGURAÇÕES DA MÁQUINA VIRTUAL (VM)

Neste *hands on*, o servidor LAMP foi originalmente implementado em máquina virtual (VM) executando o sistema operacional *Debian Linux 10 (Buster)*, conforme especificado no Quadro 1 (Especificações da VM do servidor LAMP). Portanto, as orientações à frente deverão ser devidamente adaptadas caso outra distribuição *Linux*, especialmente se não derivada do *Debian*, venha a ser utilizada na implementação. Pequenos ajustes também poderão ser necessários caso seja utilizado o *VMWare* como *hypervisor*, ao invés do *Oracle VirtualBox*.

Máquina virtual	
Recurso	Valor
Hypervisor	VirtualBox 6.1
Memória RAM	2048MB
Processadores	1
Armazenamento	Disco virtual 20GB
Rede	Modo bridge

Quadro 1 – Especificações da VM do servidor LAMP
Fonte: Elaborado pelo autor (2021)

Além disso, recomenda-se, ainda, que o *Oracle VM VirtualBox Extension Pack* também seja instalado no *hypervisor* e, por fim:

Certifique-se que o recurso de virtualização está habilitado no BIOS do *host*.

Caso o *hardware* utilizado pela máquina anfitriã (*host*) não ofereça tal recurso, é possível que a implementação do ambiente (máquina virtual) não ocorra adequadamente.

Dica de ouro que poderá lhe poupar muito tempo e ajudar na evolução dos trabalhos:

Crie um *snapshot* de sua VM a cada fase da implementação. Isto poderá evitar bastante retrabalho! Por exemplo, crie um *snapshot* da instalação inicial da VM, outro após o *APACHE* haver sido instalado com sucesso, e assim sucessivamente.

5 INSTALAÇÃO SERVIDOR APACHE

A instalação do servidor *APACHE* é muito simples: basta apenas uma única linha de comando, conforme descrito no Comando de Prompt “Instalação do servidor APACHE2”.

```
# sincroniza e atualiza pacotes
$ sudo apt-get update && sudo apt-get upgrade
# instala front-end para gerenciamento do firewall local
# instala servidor APACHE em seguida
$ sudo apt-get install ufw apache2 -y
```

Comando de prompt 1 – Instalação do servidor APACHE2

Fonte: Elaborado pelo autor (2021)

Observe que, paralelamente à instalação deste, será também instalado o pacote *ufw*: abreviação de *Uncomplicated Firewall*. Se você achava complicado trabalhar diretamente com o *iptables*, já não tem mais com o que se preocupar! Certamente, não deve ter passado por sua cabeça colocar um servidor *web* em produção sem *firewall*, não é? Ao final do processo de instalação, você poderá verificar o *status* do serviço (*APACHE*) executando a linha de comando:

```
# verifica o status do serviço HTTP
$ systemctl status apache2.service
```

Comando de prompt 2 – Verificação do status do serviço APACHE2

Fonte: Elaborado pelo autor (2021)

Se tudo estiver correto, você receberá uma mensagem com *status: active (running)* similar ao ilustrado pela Figura “Status do Apache”.

```
user1@lampsrv:~$ systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2021-02-20 10:17:52 -03; 5min ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 585 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Main PID: 607 (apache2)
     Tasks: 55 (limit: 2347)
    Memory: 11.6M
   CGroup: /system.slice/apache2.service
           └─607 /usr/sbin/apache2 -k start
             608 /usr/sbin/apache2 -k start
             609 /usr/sbin/apache2 -k start
```

Figura 3 – Status do Apache

Fonte: Elaborado pelo autor (2021)

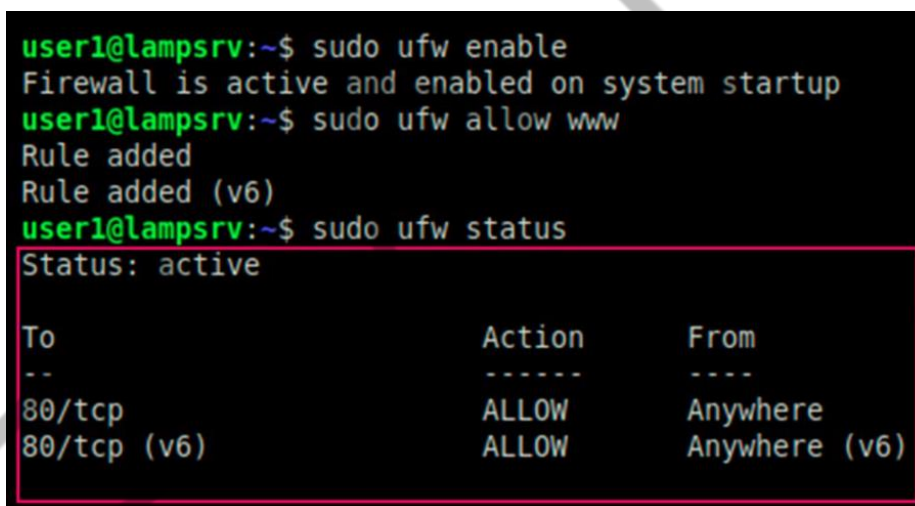
Em sequência, a ferramenta *ufw* deverá ser utilizada para configurar as regras do *firewall* que permitirão acesso ao serviço HTTP recém implementado, conforme descrito na Comando de prompt “Configuração do *firewall* local”.

```
# habilita o ufw
$ sudo ufw enable
# libera acesso ao serviço http
$ sudo ufw allow www
# verifica o status do firewall
$ sudo ufw status
```

Comando de prompt 3 – Configuração do firewall local

Fonte: Elaborado pelo autor (2021)

A Figura “Status do firewall local” ilustra a saída esperada ao se executar a sequência de comandos descritos na Comando de prompt “Configuração do *firewall* local”, quando estes forem executados com sucesso.



```
user1@lampsrv:~$ sudo ufw enable
Firewall is active and enabled on system startup
user1@lampsrv:~$ sudo ufw allow www
Rule added
Rule added (v6)
user1@lampsrv:~$ sudo ufw status
Status: active

To Action From
--
80/tcp ALLOW Anywhere
80/tcp (v6) ALLOW Anywhere (v6)
```

Figura 4 – Status do firewall local

Fonte: Elaborado pelo autor (2021)

Já na Figura “Status do firewall local” observa-se que o servidor LAMP está permitindo (*action: ALLOW*) que qualquer cliente (*from: Anywhere*) acesse seu serviço HTTP (porta 80/tcp). Como primeiro teste de funcionamento do serviço HTTP, pode-se abrir o navegador *web* no próprio servidor e digitar 127.0.0.1 ou mesmo *localhost*, na barra de endereços. Entretanto, para validação também das regras criadas pela ferramenta *ufw*, o acesso ao serviço HTTP deverá ser executado, agora, a partir do navegador da máquina real digitando-se o endereço IP do servidor LAMP na barra de endereços deste.

Caso você não se lembre, uma das formas de verificar o endereço IP em um *host* Debian é por meio do comando: `hostname -i`

Estando tudo correto, o servidor *APACHE* exibirá sua página *default* conforme ilustrado na Figura “Página *default* do APACHE”.

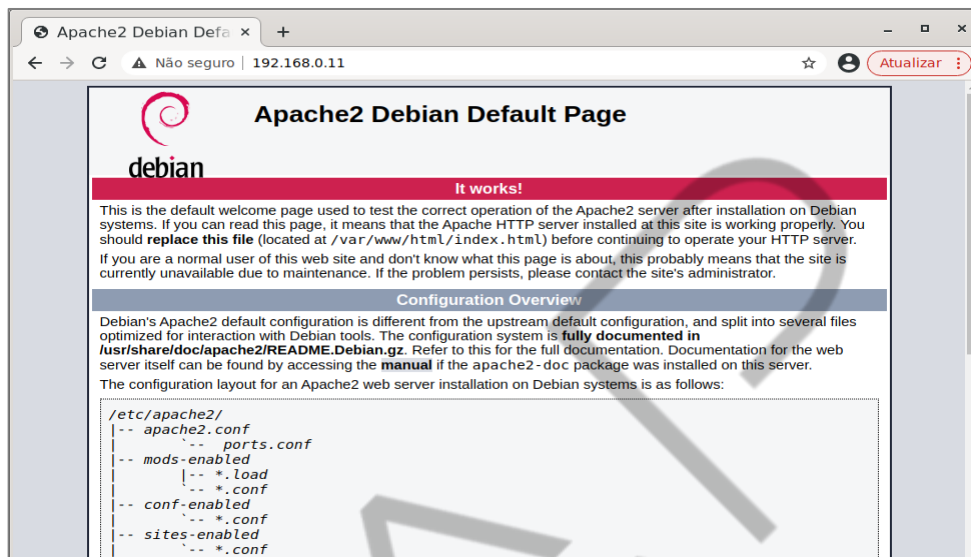


Figura 5 – Página default do Apache
Fonte: Elaborado pelo autor (2021)

6 CRIANDO VIRTUAL HOSTS

Admita que você tenha sido solicitado a implementar um *web server* o qual deverá disponibilizar múltiplos websites independentes, por exemplo, um para cada cliente de seu serviço de hospedagem web (*web hosting*). Algumas diferentes opções poderiam ser consideradas para satisfazer tal demanda:

- Hospedar cada *website* em um servidor físico independente. Provavelmente, entretanto, esta opção seria aceitável apenas em cenários bastante específicos.
- Hospedar cada *website* em uma máquina virtual independente, as quais seriam hospedadas em um único servidor físico. Embora mais plausível que a hipótese anterior, também esta opção parece ser adequada apenas a cenários bem particulares.

- Todavia, mesmo podendo-se imaginar várias outras opções para atender ao solicitado, provavelmente uma das alternativas mais satisfatórias recaia sobre a criação destes *websites* independentemente, porém, armazenados no mesmo servidor – seja este último implementado em *host* físico ou em máquina virtual.

Esta última opção sintetiza o conceito de *virtual hosts* implementado pelos *web servers* e, geralmente, representa uma solução que emprega de forma mais eficiente os recursos a serem investidos para concretização do projeto. No contexto aqui desenvolvido, esta realmente constitui a solução mais viável, porém, em casos práticos, há que se considerar, dentre outros fatores, que:

A utilização de um único servidor físico constituirá um ponto único de falha (*spof* – *single point of failure*) o qual poderá trazer sérias consequências ao negócio em caso de indisponibilidade.

Logicamente, tais consequências serão agravadas caso este *web server* hospede múltiplos *web sites*. Assim sendo, em casos reais, recomenda-se que os *web servers* sejam implementados em *clusters* de alta disponibilidade, dentre outras soluções adicionais. Em nosso contexto, entretanto, consideremos a necessidade de hospedar os *websites*: *cyberhints.com* e *cyberthreats.com* no mesmo *web server*.

Para isso, dois *virtual hosts* de mesmo nome serão criados. Em outras palavras, o termo *virtual host* se refere à prática de publicar múltiplos *web sites* em uma máquina única. *Virtual hosts* podem ser: (I) baseados em IP – quando cada *web site* possui um endereço IP distinto; ou (II) baseados em nome – o que significa que o servidor disponibiliza múltiplos nomes (“domínios”) valendo-se do mesmo endereço IP. O fato de os *virtual hosts* serem executados no mesmo servidor físico é transparente para o usuário final. Para implementação do servidor *APACHE* no contexto desenvolvido aqui, serão executados os passos a seguir.

6.1 Ajustando os módulos do APACHE

O *APACHE* utiliza vários módulos que lhe conferem funcionalidades adicionais, dentre estas, reescrever URLs nativamente para suportar criptografia SSL. Isto permite aos administradores ajustar as configurações do *web server* de forma a atender melhor às necessidades, simplesmente, adicionando ou removendo módulos.

É oferecida, ainda, uma série de módulos MPM (*Multi-Processing Modules*) responsáveis, dentre outras funções, pela vinculação às *interfaces* de rede do servidor físico e pela manipulação das requisições. Diferentemente de outros tipos de módulos, o *APACHE* permite o uso de apenas um módulo MPM por vez. A documentação oficial informa que sistemas *Unix* e *Unix-like* (*Linux*, *BSD*, *Solaris* e *Mac OS X*, dentre outros) apresentam como padrão, os módulos *prefork*, *worker* ou *event*.

Leia mais em:

<<https://httpd.apache.org/docs/2.4/mod/module-dict.html#Description>>

<<https://httpd.apache.org/docs/2.4/mpm.html>>

Especificamente no caso do *Debian*, tem-se dois módulos como padrão:

- ***mpm_prefork***: dentre outras características, é o modulo MPM mais indicado para isolar as requisições feitas ao servidor, evitando que uma venha a interferir com a outra. Assim sendo, este módulo será habilitado em nossa implementação.
- ***mpm_event***: é uma variante do MPM *worker* tendo, dentre outras funções, tentar corrigir o *keep alive* do HTTP. Este módulo será desabilitado em nossa implementação.

Acesse <<https://httpd.apache.org/docs/2.4/mod/event.html>>

Basta, para isso, executar as instruções no (Habilitando e desabilitando módulos MPM).

```
# desabilita módulo mpm_event
$ sudo a2dismod mpm_event
# habilita módulo mpm_prefork
$ sudo a2enmod mpm_prefork
# reinicializa o APACHE para efetivação das configurações
$ sudo systemctl restart apache2
# verifica módulos mpm carregados
$ sudo apachectl -M | grep mpm
```

Comando de prompt 4 – Habilitando e desabilitando módulos MPM

Fonte: Elaborado pelo autor (2021)

6.2 Criando os subdiretórios para os *virtual hosts*

O subdiretório de cada *virtual host* deverá ser criado agora, a partir de `/var/www/html`, conforme descrito na Comando de prompt “Criação dos subdiretórios do *virtual host* cyberhints.com”.

```
# cria subdiretórios para o virtual host cyberhints.com
# subdiretorio para a web page
$ sudo mkdir -p /var/www/html/cyberhints.com/public_html
# subdiretorio para logs
$ sudo mkdir /var/www/html/cyberhints.com/logs
# ajusta permissões dos subdiretórios
$ sudo chmod -R 755 /var/www/html/cyberhints.com
# ajusta propriedade dos subdiretórios
$ sudo chown -R www-data:www-data \
/var/www/html/cyberhints.com

# ATENCAO: quando da digitação do comando anterior na cli,
# ignorar a quebra de linha executada (\). Esta tem como
# função, unicamente, facilitar a leitura do código
```

Comando de prompt 5 – Criação dos subdiretórios do virtual host cyberhints.com

Fonte: Elaborado pelo autor (2021)

Cabe ressaltar que a criação dos subdiretórios para o *virtual host* cyberthreats.com deverá seguir o mesmo roteiro da referida Comando de prompt “Criação dos subdiretórios do *virtual host* cyberhints.com”, basicamente, alterando-se o nome do *virtual host*.

6.3 Configurando e ativando os *virtual hosts*

A exemplo dos subdiretórios, cada *virtual host* deverá ter seus próprios arquivos de configuração criados conforme descrito na Listagem “Arquivo de configuração do virtual host cyberhints.com”. Estes deverão ainda derivar do arquivo `000-default.conf`, sendo alocados no mesmo diretório deste: `/etc/apache2/sites-available`.

```
# cria arquivo de cfg para o virtual host cyberhints.com
$ sudo cp /etc/apache2/sites-available/000-default.conf \
/etc/apache2/sites-available/cyberhints.com.conf
# edita o arquivo de configuração
$ sudo vi \
/etc/apache2/sites-available/cyberhints.com.conf
# ajusta o conteúdo do arquivo conforme abaixo

<Directory /var/www/html/cyberhints.com/public_html>
    Require all granted
</Directory>
<VirtualHost *:8080>
    ServerName cyberhints.com
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html/cyberhints.com/public_html
    ErrorLog /var/www/html/cyberhints.com/logs/error.log
    CustomLog /var/www/html/cyberhints.com/logs/access.log \
        combined
</VirtualHost>
# ATENÇÃO: quando da digitação do comando anterior na cli,
# ignorar as quebras de linha executadas (\\). Estas têm
# como função, unicamente, facilitar a leitura do código
#
# habilitação do virtual host
$ sudo a2ensite cyberhints.com
# ativação das novas configurações
$ sudo systemctl reload apache2.service
```

Comando de prompt 6 – Arquivo de configuração do virtual host cyberhints.com

Fonte: Elaborado pelo autor (2021)

Após criado o arquivo de configuração do *virtual host* cyberhints.com, este último deverá ser habilitado conforme especificado na Listagem “Arquivo de configuração do virtual host cyberhints.com”. Se tudo estiver correto, após o *APACHE* ser recarregado o *virtual host* já estará operacional, conforme ilustrado pela Figura “Habilitação do virtual host cyberhints.com”.

```
user1@lampsrv:~$ sudo a2ensite cyberhints.com
Enabling site cyberhints.com.
To activate the new configuration, you need to run:
    systemctl reload apache2
user1@lampsrv:~$ sudo systemctl reload apache2.service
```

Figura 6 – Habilitação do virtual host cyberhints.com

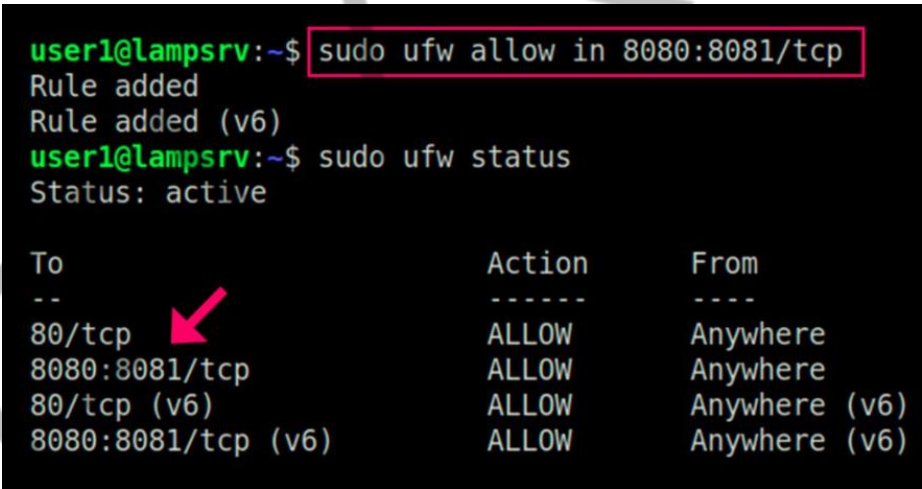
Fonte: Elaborado pelo autor (2021)

Faltam agora apenas dois passos para que se possa acessar o *virtual host*: liberar a porta 8080 no *firewall* e fazer com que o *APACHE* passe a escutar também nesta porta.

6.4 Ajustando as configurações do *firewall*

Da forma como encontra-se configurado, o *virtual host cyberhints.com* poderá ser acessado via porta 8080/TCP do servidor web. Assim sendo, sugere-se que o *virtual host cyberthreats.com* (a ser configurado por você!) seja configurado para responder às requisições enviadas à porta 8081/TCP deste servidor.

Lembrando que, como até o momento, a configuração do *firewall* local libera apenas a porta 80/TCP (Figura “Status do firewall local”), as novas configurações também deverão ser executadas conforme ilustrado pelo destaque na Figura “Liberação de portas adicionais”.



```
user1@lampsrv:~$ sudo ufw allow in 8080:8081/tcp
Rule added
Rule added (v6)
user1@lampsrv:~$ sudo ufw status
Status: active

To Action From
--
80/tcp ALLOW Anywhere
8080:8081/tcp ALLOW Anywhere
80/tcp (v6) ALLOW Anywhere (v6)
8080:8081/tcp (v6) ALLOW Anywhere (v6)
```

Figura 7 – Liberação de portas adicionais
Fonte: Elaborado pelo autor (2021)

6.5 Ajustando as configurações de portas do *APACHE*

Por padrão, o *APACHE* só escuta requisições encaminhadas às portas 80/TCP (HTTP) e 443/TCP (HTTPS). Portanto, será necessário editar o arquivo `/etc/apache2/ports.conf`, e ajustar as suas configurações, conforme os destaques no Código-fonte “Ajustes nas portas do *APACHE*”.

```
# If you just change the port or add more ports here, you
will likely also
# have to change the VirtualHost statement in
# /etc/apache2/sites-enabled/000-default.conf

Listen 80
Listen 8080
Listen 8081

<IfModule ssl_module>
    Listen 443
</IfModule>

<IfModule mod_gnutls.c>
    Listen 443
</IfModule>

# vim: syntax=apache ts=4 sw=4 sts=4 sr noet
```

Código-fonte 1 – Ajustes nas portas do Apache
Fonte: Elaborado pelo autor (2021)

Finalizada a edição do arquivo, basta mais uma vez recarregar as configurações do *APACHE* (`$ sudo systemctl reload apache2.service`) e verificar se as portas 8080 e 8081 estão aguardando conexões: *state LISTEN* (`$ss -nat`), conforme ilustrado pela Figura “Verificação dos estados das portas”.

```
user1@lampsrv:~$ sudo systemctl reload apache2.service
user1@lampsrv:~$ ss -nat
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port
LISTEN	0	128	*:8080	*:*
LISTEN	0	128	*:80	*:*
LISTEN	0	128	*:8081	*:*
TIME-WAIT	0	0	::ffff:192.168.0.11]:80	::ffff:192.168.0.7]:54322
FIN-WAIT-2	0	0	::ffff:192.168.0.11]:80	::ffff:192.168.0.7]:54326
TIME-WAIT	0	0	::ffff:192.168.0.11]:80	::ffff:192.168.0.7]:54320

Figura 8 – Verificação dos estados das portas
Fonte: Elaborado pelo autor (2021)

Para testar o funcionamento do *virtual host*, crie um arquivo chamado `index.html` no interior do diretório `/var/www/html/cyberhints.com/public_html/` com o conteúdo sugerido na Listagem 8 (Página para testes - cyberhints.com).

```
<html>
  <head>
    <title>Bem-vindos a CyberHints</title>
    <meta charset="utf-8"/>
  </head>
  <body>
    <h1>Bem-vindo a CyberHints</h1>
    <h2>Aproveite para estudar HTML também!!</h2>
  </body>
</html>
```

Código-fonte 2 – Página para testes - cyberhints.com
Fonte: Elaborado pelo autor (2021)

Estando tudo certo, acesse seu servidor utilizando [IP_SERVER]:8080, conforme ilustrado pela Figura “Página para testes - cyberhints.com” e poderá ver seu *virtual host* em ação.

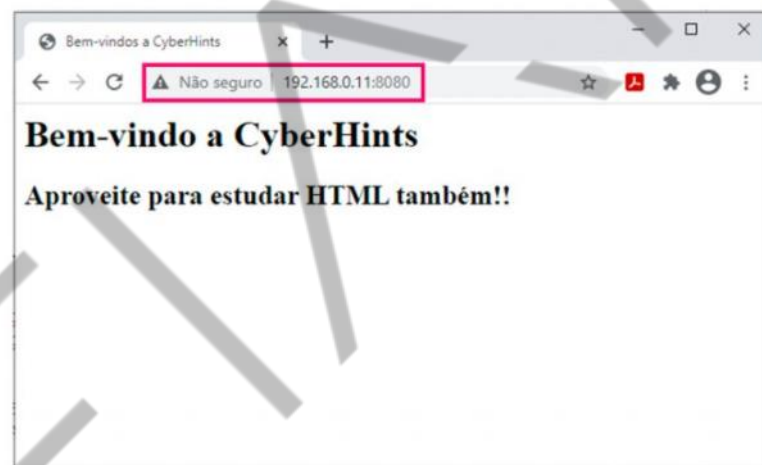


Figura 9 – Página para testes - cyberhints.com
Fonte: Elaborado pelo autor (2021)

7 INSTALAÇÃO DO MARIADB

O *MariaDB* é um *fork* do *MySQL* e sua primeira versão foi lançada em 1995, inicialmente para uso pessoal. Entretanto, ele logo obteve grande destaque e foi adotado também pelo mundo corporativo, por conta das vantagens que apresentava em relação à formação e gerenciamento de bancos de dados. Em 2008, o *MySQL* foi comprado pela *Sun Microsystems* a qual, por sua vez, foi comprada pela *Oracle*.

Todavia, a maior parte dos desenvolvedores originais não gostou da gestão da *Oracle* e acabaram se dividindo, dando início ao *MariaDB*, em 2009. Como no início do *MySQL*, o *MariaDB* também é um *software open source* e colaborativo, porém, o novo modelo garante mais clareza em relação ao fluxo de *patches* adicionados. As decisões acerca do desenvolvimento são publicadas em lista para revisão que pode ser discutida publicamente no *Public Bug Tracker*.

O *MariaDB* tem melhor desempenho que o *MySQL* dado seu método de busca otimizado, sendo a migração para este (a partir do *MySQL*) bastante tranquila. Ambos trabalham com os mesmos tipos de dados e a mesma linguagem PL/SQL, ou seja, os comandos empregados pelos dois são idênticos.

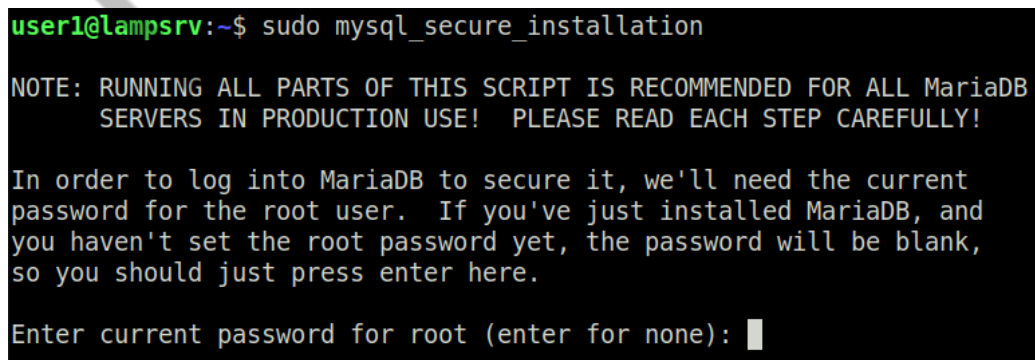
A instalação do *MariaDB* é muito simples, conforme descrito no Comando de prompt “Instalação do *MariaDB*”

```
# instala o servidor mariadb  
$ sudo apt-get install mariadb-server -y
```

Comando de prompt 7 – Instalação do *MariaDB*
Fonte: Elaborado pelo autor (2021)

Finalizada a instalação do *MariaDB* deverá, então, ser executado o *script mysql_secure_installation* a fim de alterar algumas opções padrão menos seguras inicialmente implementadas. A execução deste *script* é bastante simples e requer que algumas configurações básicas sejam executadas durante o processo.

Na Figura “Senha atual do root” é possível observar que a primeira informação solicitada é a senha corrente do usuário *root*.



```
user1@lampsrv:~$ sudo mysql_secure_installation  
  
NOTE: RUNNING ALL PARTS OF THIS SCRIPT IS RECOMMENDED FOR ALL MariaDB  
SERVERS IN PRODUCTION USE! PLEASE READ EACH STEP CAREFULLY!  
  
In order to log into MariaDB to secure it, we'll need the current  
password for the root user. If you've just installed MariaDB, and  
you haven't set the root password yet, the password will be blank,  
so you should just press enter here.  
  
Enter current password for root (enter for none):
```

Figura 10 – Senha atual do root
Fonte: Elaborado pelo autor (2021)

Além disso, é possível observar também que a senha inicial solicitada deverá ficar em branco, portanto, simplesmente tecele *Enter* para continuar. Em seguida é, então, solicitada a nova senha do *root*, conforme ilustrado pela Figura “Troca da senha do root”.

```
Enter current password for root (enter for none):
OK, successfully used password, moving on...

Setting the root password ensures that nobody can log into the MariaDB
root user without the proper authorisation.

Set root password? [Y/n] Y
New password:
Re-enter new password:
Password updated successfully!
Reloading privilege tables..
... Success!
```

Figura 11 – Troca da senha do root
Fonte: Elaborado pelo autor (2021)

Por padrão, a instalação do *MariaDB* cria um usuário anônimo o qual deve ser removido por motivos segurança, conforme observado na mensagem da Figura (Remoção de usuários anônimos). Assim sendo, ao receber a mensagem ilustrada pela referida figura, basta digitar [Y] ou *Enter* para confirmar a ação.

```
By default, a MariaDB installation has an anonymous user, allowing anyone
to log into MariaDB without having to have a user account created for
them. This is intended only for testing, and to make the installation
go a bit smoother. You should remove them before moving into a
production environment.

Remove anonymous users? [Y/n] ☒
```

Figura 12 – Remoção de usuários anônimos
Fonte: Elaborado pelo autor (2021)

Ainda por motivos de segurança, o *script* recomenda agora que o acesso administrativo ao banco de dados por parte do usuário *root* seja feito apenas localmente. Assim sendo, aceitaremos a opção *default* mais uma vez digitando [Y] ou *Enter*, conforme demonstrado pela Figura “Desabilitando o login remoto”.

```
Normally, root should only be allowed to connect from 'localhost'. This
ensures that someone cannot guess at the root password from the network.

Disallow root login remotely? [Y/n] Y
... Success!
```

Figura 13 – Desabilitando o login remoto
Fonte: Elaborado pelo autor (2021)

Por fim, deve-se ainda remover o *database test* que também acompanha o *MariaDB* e remover seus privilégios, mais uma vez aceitando as opções *default* oferecidas pelo *script*, conforme ilustrado pela Figura “Remoção do database test”. Isto feito, a instalação inicial do *MariaDB* estará finalizada.

```
By default, MariaDB comes with a database named 'test' that anyone can
access. This is also intended only for testing, and should be removed
before moving into a production environment.

Remove test database and access to it? [Y/n] Y
- Dropping test database...
... Success!
- Removing privileges on test database...
... Success!

Reloading the privilege tables will ensure that all changes made so far
will take effect immediately.

Reload privilege tables now? [Y/n] Y
... Success!

Cleaning up...

All done! If you've completed all of the above steps, your MariaDB
installation should now be secure.

Thanks for using MariaDB!
```

Figura 14 – Remoção do database test

Fonte: Elaborado pelo autor (2021)

Para verificar o *status* do serviço, basta executar o comando: *systemctl status mariadb.service*, conforme demonstrado na Figura “Verificando o status do serviço MariaDB”.

```
user1@lampsrv:~$ systemctl status mariadb.service
● mariadb.service - MariaDB 10.3.27 database server
   Loaded: loaded (/lib/systemd/system/mariadb.service; enabled; vendor preset: enabled)
   Active: active (running) since Tue 2021-02-23 22:14:17 -03; 1h 1min ago
     Docs: man:mysqld(8)
           https://mariadb.com/kb/en/library/systemd/
  Main PID: 2757 (mysqld)
    Status: "Taking your SQL requests now..."
     Tasks: 30 (limit: 2347)
    Memory: 76.5M
   CGroup: /system.slice/mariadb.service
           └─2757 /usr/sbin/mysqld
```

Figura 15 – Verificando o status do serviço MariaDB

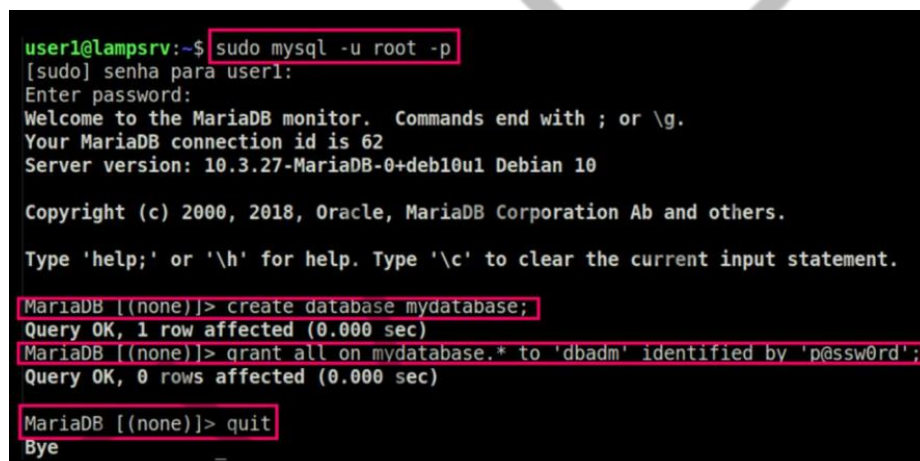
Fonte: Elaborado pelo autor (2021)

Pode-se agora criar uma *database* e atribuir as permissões para que os usuários possam acessá-la. Para tal, basta seguir os comandos no Comando de prompt “Configuração do MariaDB”.


```
# fazer login no MariaDB como root
$ sudo mysql -u root -p
# fornecer a senha do root no sistema e no MariaDB
# criar a database mydatabase
MariaDB [(none)]> create database mydatabase;
# atribuir permissões ao usuário dbadm / senha p@ssw0rd
MariaDB [(none)]> grant all on mydatabase.* to 'dbadm'
identified by 'p@ssw0rd';
```

Comando de prompt 8 – Configuração do *MariaDB*
Fonte: Elaborado pelo autor (2021)

A Figura “Configuração do *MariaDB*” ilustra a sequência descrita na Comando de prompt “Configuração do *MariaDB*”.



```
user1@lampsrv:~$ sudo mysql -u root -p
[sudo] senha para user1:
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 62
Server version: 10.3.27-MariaDB-0+deb10u1 Debian 10

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> create database mydatabase;
Query OK, 1 row affected (0.000 sec)
MariaDB [(none)]> grant all on mydatabase.* to 'dbadm' identified by 'p@ssw0rd';
Query OK, 0 rows affected (0.000 sec)

MariaDB [(none)]> quit
Bye
```

Figura 16 – Configuração do *MariaDB*
Fonte: Elaborado pelo autor (2021)

Chegando a este ponto, a implementação do MariaDB já estará pronta para uso!

DICA: Uma boa ferramenta gráfica para auxiliar na administração do *MariaDB* (assim como outros gerenciadores de bancos de dados) é o *DBeaver Community Edition*, disponível em: <https://dbeaver.io/download/>

8 INSTALAÇÃO DO PHP

PHP (acrônimo recursivo para PHP: *Hypertext Preprocessor*) é uma linguagem de *script* de uso geral, *open source*, bastante difundida para desenvolvimento *web* e que pode ser embutida em blocos de código HTML. A Listagem 11 fornece um exemplo bastante simples de código PHP.

```
<!DOCTYPE HTML>
<html>
  <head>
    <title>Exemplo</title>
  </head>
  <body>
    <?php
      echo "Olá, eu sou um script PHP!";
    ?>
  </body>
</html>
```

Código-fonte 3 – Exemplo de código PHP
Fonte: PHP (2021)

Observe que o PHP mantém a simplicidade do código HTML original da página se mesclando a este. No Código-fonte “Exemplo de código PHP” é possível observar que o bloco de código PHP é delimitado por *tags* próprias que permitem transitar facilmente entre o bloco HTML e o bloco PHP. O PHP é uma linguagem *server side*, ou seja, é executado no lado do servidor renderizando o código HTML enviado ao navegador do cliente. Portanto, o cliente recebe o resultado da execução do código PHP sem tomar conhecimento do código-fonte deste.

A instalação do PHP no servidor LAMP também é tarefa bastante simples, bastando para isso a execução do comando ‘`sudo apt-get install php7.3 libapache2-mod-php7.3 php-mysql -y`’. Observe que tal linha de comando instala também as peças de *software* necessárias à integração do PHP com o *APACHE* e *MySQL*. À época do desenvolvimento deste material, a versão do PHP mantida nos repositórios *Debian* era a 7.3, não sendo necessariamente a mesma disponível quando você instalar. A Figura “Instalação do PHP” ilustra parcialmente o processo de instalação do PHP.

```
user1@lampsrv:~$ sudo apt-get install php7.3 libapache2-mod-php7.3 php-mysql -y
[sudo] senha para user1:
Lendo listas de pacotes... Pronto
Construindo árvore de dependências
Lendo informação de estado... Pronto
The following additional packages will be installed:
  php-common php7.3-cli php7.3-common php7.3-json php7.3-mysql php7.3-opcache
  php7.3-readline
Pacotes sugeridos:
  php-pear
Os NOVOS pacotes a seguir serão instalados:
  libapache2-mod-php7.3 php-common php-mysql php7.3 php7.3-cli php7.3-common php7.3-json
  php7.3-mysql php7.3-opcache php7.3-readline
0 pacotes atualizados, 10 pacotes novos instalados, 0 a serem removidos e 1 não atualizados.
É preciso baixar 4.137 kB de arquivos.
Depois desta operação, 18,0 MB adicionais de espaço em disco serão usados.
```

Figura 17 – Instalação do PHP
Fonte: Elaborado pelo autor (2021)

A configuração do PHP deverá ser feita conforme descrito no Comando de prompt “Configuração básica do PHP”.

```
# fazer cópia arquivo php.ini antes d editá-lo
# a quebra de linha (\) é apenas para melhor visualiza-ção
# da linha de comando, não devendo ser realizada na cli
$ sudo cp /etc/php/7.3/apache2/php.ini \
/etc/php/7.3/apache2/php.ini.bkp
# editar o arquivo php.ini com a ferramenta que mais
# lhe convier
$ sudo vi /etc/php/7.3/apache2/php.ini
# criar o diretório para o PHP com permissões e posse
# do usuário www-data (utilizado pelo APACHE)
$ sudo mkdir /var/log/php
$ sudo chown www-data /var/log/php
# inicializar o serviço apache2
$ sudo systemctl restart apache2.service
```

Comando de prompt 9 – Configuração básica do PHP

Fonte: Elaborado pelo autor (2021)

A fim de testar o funcionamento do PHP com o *MariaDB*, siga o procedimento descrito na Listagem “Arquivo index.php”. O PHP é uma linguagem bastante intuitiva e, mesmo com pouca familiaridade, o entendimento do código na referida listagem não será difícil.

```
# renomear o arquivo index.html no diretório public_html
# do virtual host cyberhints.com, lembrando que a quebra
# de linha abaixo (\) não deve ser feita na cli
$ sudo mv /var/www/html/cyberhints.com/public_html/index.html
\
/var/www/html/cyberhints.com/public_html/index.000
# criar o arquivo index.php no mesmo diretório
$ sudo vi /var/www/html/cyberhints.com/public_html/index.php
```

Comando de prompt 10 – Acesso ao index.php

Fonte: Elaborado pelo autor (2021)

```
#copiar o fragmento de código abaixo e colar no referi-do
#arquivo
#
<html>
<head>
    <title>Bem-vindos a CyberHints</title>
    <meta charset="utf-8"/>
</head>
<body>
    <h1>Bem-vindo a CyberHints</h1>
    <h2>Aproveite para estudar HTML também!!</h2>
    <?php
        $srvname = "localhost";
        $username = "dbadm";
        $usrpwd = "p@ssw0rd";
        $dbname = "mydatabase";

        $conn = mysqli_connect($srvname , $username , $usrpwd ,
        $dbname);
        echo "Tentando estabelecer conexao com MariaDB<br>";

        if (!$conn){

            echo "<b>Erro de conexao com o banco de dados</b>";

        }else{

            echo "<b>Banco de dados conectado com sucesso</b>";

        }
    ?>
</body>
</html>
```

Código-fonte 4 – Arquivo index.php acessando o MariaDB

Fonte: Elaborado pelo autor (2021)

Antes de encerrarmos, talvez você ainda esteja um pouco desconfortável em relação à administração do banco de dados. Então, indo um pouco além do combinado, instalemos agora o *PHPMyAdmin*.

Trata-se de uma ferramenta em software livre e de código-aberto desenvolvida em PHP para administração do *MySQL* via *Internet*, mas também pode ser usado localmente sem nenhum problema.

Esta ferramenta tem instalação e configuração básica bastante simples e, dentre outras funcionalidades, possibilita a criação e remoção de *databases*, tabelas e execução de códigos SQL.

O projeto original do *PHPMyAdmin* pode ser acessado no endereço <<https://www.phpmyadmin.net/>> , onde há rico material a respeito dele. Em nosso caso, entretanto, a instalação será feita via repositórios do *Debian*. No entanto, até o momento da finalização deste documento, esta ferramenta não era diretamente disponibilizada nos repositórios do *Debian10*, razão pela qual o arquivo */etc/apt/sources.list* – responsável pelos repositórios disponibilizados ao *host* local –, terá de ser devidamente ajustado, bastando, para isto, executar os comandos descritos na Listagem 14 (Inserção de repositórios).

```
# criar cópia de segurança do arquivo sources.list
$ sudo cp /etc/apt/sources.list /etc/apt/sources.list.bkp
# abrir um editor de textos com privilégios de root e
$ sudo vi /etc/apt/sources.list
# inserir os repositórios adicionais necessários à
# instalação do PHPMyAdmin
# repositório 1:
# deb http://deb.debian.org/debian/ buster-backports main
# repositório 2:
# deb-src http://deb.debian.org/debian/ buster-backports main
# após inseridos os novos repositórios, atualizar a lista
# de pacotes
$ sudo apt-get update
# instalar o PHPMyAdmin
$ sudo apt install -t buster-backports phpMyAdmin
# desta vez foi, realmente, utilizado o comando apt ao
# invés do tradicional apt-get
```

Comando de prompt 11 – Inserção de repositórios
Fonte: Elaborado pelo autor (2021)

Durante o processo de instalação, o *phpMyAdmin* fará alguns questionamentos simples.

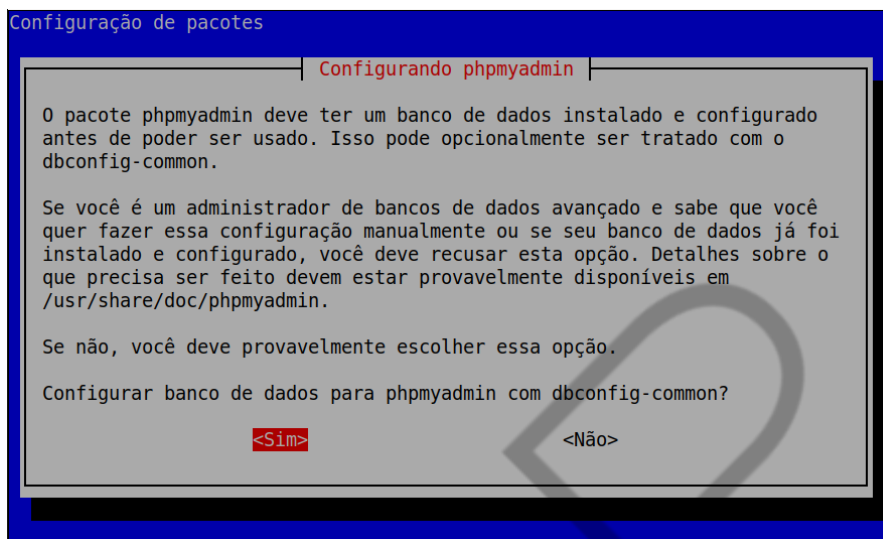


Figura 18 – Configurar banco de dados
Fonte: Elaborado pelo autor (2021)

Na fase ilustrada pela Figura “Configurar banco de dados”, responda sim ao ser questionado acerca da configuração do banco de dados. Na sequência, será solicitada uma senha para o *phpMyAdmin*.

No contexto aqui desenvolvido, não há necessidade de uma senha forte, portanto, coloque a senha que melhor lhe convier (Figura “Senha para o phpmyadmin”), confirmando esta senha na tela que surgirá na sequência. Alguns segundos depois a instalação estará finalizada.

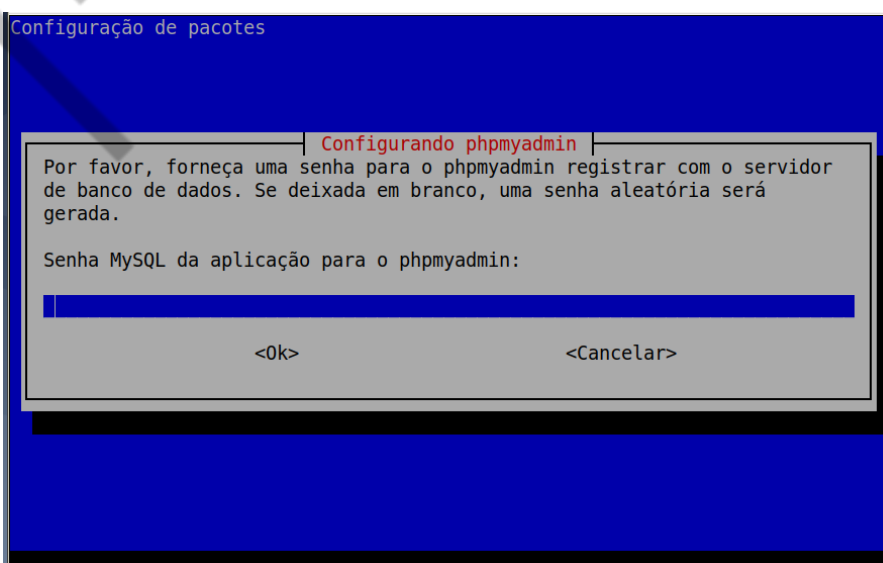


Figura 19 – Senha para o phpMyAdmin
Fonte: Elaborado pelo autor (2021)

Para utilizar a ferramenta, basta abrir o navegador e, na barra de endereços, digitar: '<http://localhost/phpmyadmin/>' e a tela de boas-vindas da ferramenta surgirá, conforme ilustrado pela Figura “Boas-vindas PHPMyAdmin”.

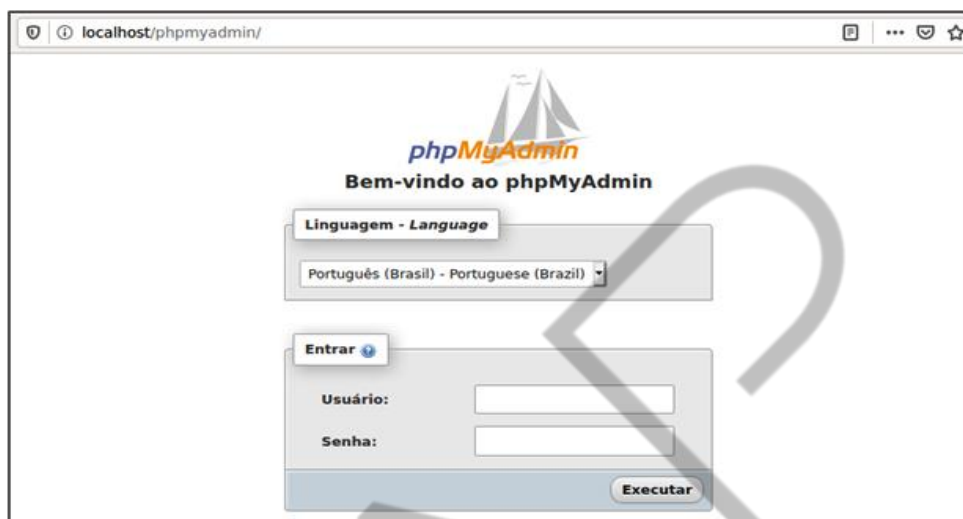


Figura 20 – Boas-Vindas PHPMyAdmin
Fonte: Elaborado pelo autor (2021)

Observe que as credenciais solicitadas para *login* na ferramenta são o *username* e *password* de usuário já previamente cadastrado no *MariaDB*, no caso, portanto, username: *dbadm* / senha: *p@ssw0rd*. A Figura “Painel principal do PHPMyAdmin” ilustra o local em que é possível verificar o banco de dados anteriormente criado (*mydatabase*).

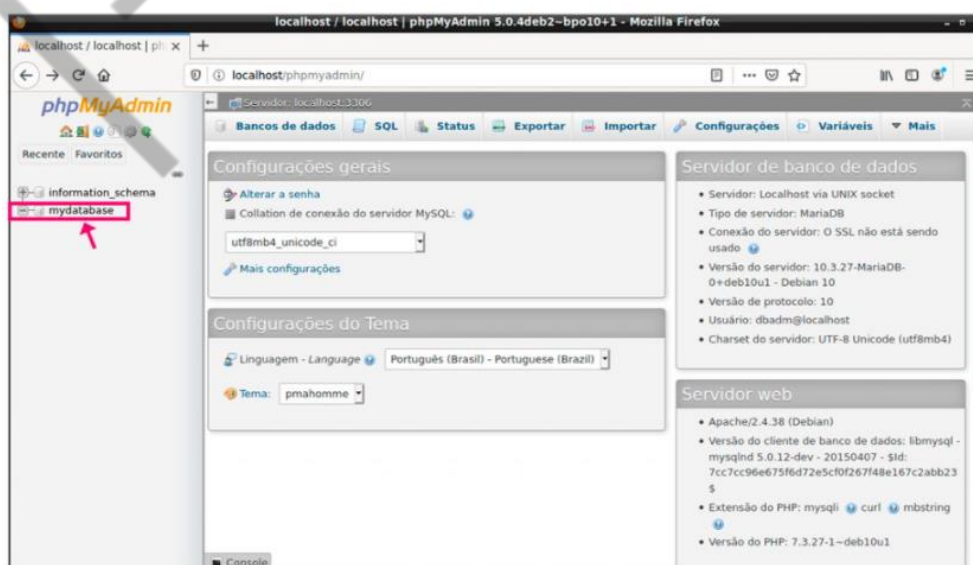


Figura 21 – Painel principal do PHPMyAdmin
Fonte: Elaborado pelo autor (2021)

Assim, chegamos ao final deste capítulo!

Esperamos que o seu interesse por linguagens de programação e banco de dados tenha sido despertado, pois ambos são altamente relevantes para uma formação mais completa e sólida.

Entretanto, como o escopo deste projeto não contempla o devido aprofundamento nestes temas, consulte as referências disponibilizadas ao final do capítulo para mais informações, e não se esqueça que dedicação é fundamental!

EMANUEL

REFERÊNCIAS

9 VANTAGENS de migrar do MySQL para o MariaDB. **EVEO**, 2018. Disponível em: <<https://www.eveo.com.br/blog/vantagens-mariadb/>>. Acesso em: 5 mar. 2021.

APACHE. [s.d.]. Disponível em: <<http://www.apache.org/>>. Acesso em: 5 mar. 2021.

APACHE. **Apache Virtual Host Documentation**. [s.d.]. Disponível em: <[https://httpd.apache.org/docs/2.4/vhosts/#:~:text=The%20term%20Virtual%20Host%20refers,com%20\)%20on%20a%20single%20machine.&text=Apache%20was%20one%20of%20the,right%20out%20of%20the%20box](https://httpd.apache.org/docs/2.4/vhosts/#:~:text=The%20term%20Virtual%20Host%20refers,com%20)%20on%20a%20single%20machine.&text=Apache%20was%20one%20of%20the,right%20out%20of%20the%20box)>. Acesso em: 10 fev. 2021.

APACHE. **Multi-processing Modules (MPMs)**. [s.d.]. Disponível em: <<https://httpd.apache.org/docs/2.4/mpm.html>>. Acesso em: 10 fev. 2021.

BRINGING MySQL to the web. **Phpmyadmin**, [s.d.]. Disponível em: <<https://www.phpmyadmin.net/>>. Acesso em: 15 fev. 2021.

LEWIS, S. **How To Setup a Firewall with UFW on na Ubuntu and Debian Cloud Server**. 2013. Disponível em <<https://www.digitalocean.com/community/tutorials/how-to-setup-a-firewall-with-ufw-on-an-ubuntu-and-debian-cloud-server>>. Acesso em: 5 mar. 2021.

MARIADB. **Training & Tutorials**. [s.d.]. Disponível em: <<https://mariadb.com/kb/en/training-tutorials/>>. Acesso em: 5 mar. 2021.

MARIADB. **MariaDB Tutorial**. [s.d.]. Disponível em: <<https://www.mariadbtutorial.com/>>. Acesso em: 5 mar. 2021.

O QUE É O PHP? **PHP**, [s.d.]. Disponível em: <https://www.php.net/manual/pt_BR/intro-what-is.php>. Acesso em: 12 fev. 2021.

O QUE É software livre? [s.d.]. Disponível em: <<https://www.gnu.org/philosophy/free-sw.pt-br.html>>. Acesso em: 5 mar. 2021.

PHP TUTORIAL for Beginners: Learn in 7 Days. **Guru99**, [s.d.]. Disponível em: <<https://www.guru99.com/php-tutorials.html>>. Acesso em: 5 mar. 2021.

STACKSHARE. **Apache HTTP Server vs Cherokee vs NGINX**. 2021. Disponível em: <<https://stackshare.io/stackups/apache-httpd-vs-cherokee-vs-nginx>>. Acesso em: 3 mar. 2021.

THREAD pool. **Percona**. [s.d.]. Disponível em: <<https://www.percona.com/doc/percona-server/8.0/performance/threadpool.html>>. Acesso em: 5 mar. 2021.